Computational Nonlinear Dynamics (MTH3039, 2026/27 Term 1)
Coursework 1

Module lead: George Datseris

This coursework is worth 50% of the credits for this module. The maximum number of marks for this coursework is 100, separated into into thematic task groups.
Deadline for this coursework is Friday 6th of November, 12:00.

Individual coursework

This is an **individual coursework** and your attention is drawn to the University guidelines on collaboration and plagiarism available in the University's Teaching Quality Assurance Manual:
https://www.exeter.ac.uk/v8media/specificsites/tqa/apa/APA_Handbook_Chapter_12.pdf.

Usage of AI

This assessment is AI-Minimal. This means that you can only use AI-assisted tools to check your spelling and/or grammar. For no other reason whatsoever. This includes using AI for summaries, including the search engine summaries such as "Google AI Summary". The computer lab demos are your opportunity to demonstrate that you truthfully followed these rules.

Submitting your work

Submission is done via ELE. You must submit a `.zip` file containing (in the following names replace 'X' with the coursework number, like 1, 2, ...):

- A Jupyter notebook `CWX.ipynb`. The Jupyter notebook contains the solutions for all tasks: code, code output, and interpretation statements. You can view an example submitted Jupyter notebook in the same ELE tile as the coursework briefs; use it as your starting point.
  - **The notebook must be already run so that all code is accompanied with its numeric or visualized output, and text-based explanations and comments are appended where necessary. Sections/tasks without output (or with errors for output) are graded 0 regardless of source code.**
  - The notebook **must** be structured in sections corresponding to the individual task numbers in the coursework brief (see example notebook).
  - The notebook **must not** repeat the coursework brief itself, but proceed straight to the answers.
  - The notebook **must** contain the line `include("CWX.jl")` as its first code block.
- A Julia source code file `CWX.jl` that contains function definitions that you use in the above Jupyter notebook. The file `CWX.jl` **must** contain definitions for all named functions mentioned in this coursework, e.g., a function named `psos1` or `fg1`, etc.. Once `CW_X.jl` is run, all named functions mentioned in this coursework **must** be accessible.
- (Optional) Any other Julia source code files that you `include()` in the above file, in case you want to organize your codebase into several files (which you are allowed to do).
- (Optional) Any binary files that you saved to disk while working on the coursework, see the discussion below on `JLD2`.

Learning outcomes

This coursework assesses all learning outcomes listed in the module descriptor.

Marking criteria

- 100%: Jupyter notebook output (typically a figure) is fully correct. Output is interpreted and discussed in sufficient depth and with correct insight. Output is visually clear: different things are coloured differently or are plotted with different markers or linestyles, etc. Jupyter code (and therefore also source code) runs and creates the same figures as saved in the Jupyter notebook. Code is generalizable: if applied to a different dynamical system with appropriate inputs it will work as intended.
- 70-99%: Output is mostly correct but it has minor mistakes that can be corrected with minor work. Interpretation of the output is mostly correct or with insufficient depth. Some of the source code functions are not generalizable.
- 40-70%: Output is not correct, but there is some effort that is related with the given task, and the code runs. The code can be corrected with extensive work to provide correct output. Interpretation is incorrect at times.
- 0-39%: Most or all output is missing or invalid (i.e., code output is just error messages or it does not make any sense). Most or all interpretation is missing or is unintelligible.
- Potential for small penalty of negative points: being "confidently incorrect". For example, when listing figures that are obviously wrong, because e.g., they contradict previous figures or statements, yet boldly stating that they are "correct" and "confirm" how everything is perfect. If your figures are incorrect you don't have to say anything, or even better acknowledge that something is wrong.

A note on grading: we follow a deductive grading approach. We start by assuming you scored a grade of 100, and proceed to deduct points for incorrect tasks. Unless in exceptional circumstances, only deductions will be stated; everything not deducted was solved perfectly correctly.

Restrictions on used packages

Throughout this coursework you have to use parts of the DynamicalSystems.jl library. You can create dynamical systems (via the `CoupledODEs` or `DeterministicIteratedMap` functions). You can use the `trajectory, step!` functions at will. You can use the querying and altering interface for `DynamicalSystem` such as the functions `set_state!`, `reinit!`, or `set_parameter!` (in general, all functions described in the documentation of the `DynamicalSystem` type). **Unless explicitly allowed in the coursework brief, other components of DynamicalSystems.jl are forbidden from your submitted solutions**. However, you may use the entirety of DynamicalSystems.jl *outside* your submitted solutions to check whether your solutions are correct or in general to enhance your understanding.

For plotting you must use the `CairoMakie` package. You may use `JLD2` to save data to binary files that you can load in later sections of the notebook. This allows you to perform an extensive computation (such as computing the basins of attraction), then save it to disk, and then resume your work later by loading this file in. You must also include such binary files in your submission if you chose this approach. `JLD2` is also used to load data provided in the coursework brief. All packages belonging to the Julia standard library (those installed with Julia, such as `LinearAlgebra` or `Statistics` or anything else listed in the official Julia documentation at docs.julialang.org) may be used as well.

No other packages or functionality may be used. If any is, the coursework will be rendered invalid and receive no grade.

1 Basics of dynamical systems [25 points]

1.1 (2 points) Consider the following dynamical system

$$\begin{aligned} \dot{x} &= -y - z \\ \dot{y} &= x + ay \\ \dot{z} &= b + z(x - c) \end{aligned} \tag{1}$$

where a, b, c are all ≥ 0 parameters. Analytically calculate the fixed points of the system.

1.2 (3 points) Analytically calculate the stability of the fixed points using the Jacobian, which you also have to derive analytically. List the parameter ranges for which each fixed point is stable and unstable. For which parameter combinations, if any, is any of the fixed points stable?

1.3 (3 points) Create a `DynamicalSystem` corresponding to Eqs. (1). Set $a = 0.2, c = 5.7$ and consider the following values for b: [1.6, 0.8, 0.2]. For each b value simulate and visualize in a 3D axis the system trajectory starting always from the initial condition $[5.2, 0, 0]$. What type of trajectories do you observe? *Hint: Play around with the transient and integration times to ensure you have converged to the attractor of the system. Use a sampling time of* $\Delta t = 0.05$.

1.4 (2 points) Consider the range `b = 0.2:0.01:1.6`. For each value of `b` in this range, create a trajectory of the system as in the previous task and store the average of the x- and z- timeseries. Plot these averages versus `b`.

1.5 (3 points) Using the Jacobian, and given that $a = 0.2, c \geq 5$, argue why the system of Eq. (1) is dissipative. You can assume that the results of the previous task persist for any value of b.

1.6 (6 points) Create your own function `psos1(A, index, value)`, which estimates the Poincaré section of a trajectory `A` with a hyperplane defined for when a variable of the trajectory (with given `index`) crosses a specific `value` from negative to positive. Your function should use interpolation as discussed in the lectures.

Set $b = 0.2$ and simulate a trajectory with the initial condition $[5.2, 0, 0]$ and total integration time of $T = 1000$. Consider the $x - z$ plane defined by $y = 0$. Use your function to calculate the Poincaré section with this plane. Create a figure with two panels. On the left panel, plot the trajectory as a line plot on a 3D axis and scatterplot the Poincaré section points on top. On the right panel scatterplot the Poincaré section points on the $x - y$ plane.

1.7 (4 points) Modify your function to `psos2(A, tvec, index, value)` with the additional argument `tvec` being the time vector corresponding to trajectory `A`. Your new function should now also calculate the return times to the Poincaré section. Apply this function to the trajectory of the previous task. Show the return times vector as a result for this task.

1.8 (2 points) Plot the timeseries of the y coordinate of the trajectory you have been using so far, but limit the time axis to (10, 20) for visual clarity. Given this timeseries plot, what do you expect the average return time to be? Plot a histogram of the return times that you have calculated in the previous task and see if it confirms your intuition.

2 Multistability [25 points]

2.1 (2 points) Consider the following dynamical system

$$\begin{aligned} \dot{N} &= rN(1 - (c/r)N)(\tfrac{N-\mu}{N+\nu}) - s \\ \dot{P} &= \chi s - \delta P \\ \text{where} \quad s &= \alpha NP/(\beta + N) \end{aligned} \tag{2}$$

with parameters of interest $r = 2, \delta = 2.2$. The rest of the parameters remain fixed as $c, \mu, \mu, \alpha, \beta, \chi = 0.19, 0.03, 0.003, 800, 1.5, 0.004$. The system is a predator prey model for populations of prey N and predator P. At the given parameters, this system is multistable.

This dynamical system has two coexisting attractors (for the starting parameters). Find them, not with any formal method, but just by eye, by plotting trajectories of different initial conditions that end up at different attractors. Show a plot of at least two trajectories, each going to a different attractor.

2.2 (3 points) Create your own grouping function `simple_grouping(features, r)` that you will later use to estimate basins of attraction. The function inputs a vector of vectors and a real number (the threshold) and outputs the vector of group indices that each feature belongs in. It implements the simple grouping algorithm discussed in the lectures. Test your function by generating random data using the following code snippet:

```
randompoint() = [rand([-1, 1]) + 0.1randn(), 0.1randn()]
features = [randompoint() for _ in 1:1000]
```

Your function should group these data into two groups for an appropriate `r`. Show this by plotting the data colour-coded by the group they belong in.

2.3 (5 points) Create your own general purpose "featurize and group" function for mapping initial conditions into their basins. Name your function `fg1(ds, featurizer, r, ics; Ttr, Dt, T)`. The input arguments are an instance of a `DynamicalSystem`, a featurizing function (see below), a feature distance threshold, and an iterable of initial conditions `ics`. The function returns the group labels of the `ics`. Inside the main function, each initial condition generates a trajectory generates a trajectory using `trajectory(ds, T, u0; Ttr, Dt)`. For this predator prey model use always `Ttr = 400, T = 40, Dt = 0.2`. Then, cast each trajectory `X` into a feature-vector using the `featurizer` input, which is itself a function that takes as an input a trajectory (vector of state space points) and outputs a feature vector. You need to decide for yourself what `featurizer, r` should be so that all attractors are separated into distinct groups correctly.

Use your `fg1` function to calculate and plot the basins of attraction (BoA) of the predator prey model. To do so, initialize a grid of initial conditions given by

```
density = 100
Ng = range(0, 15; length = density) .+ 0.01
Pg = range(0, 30e-3; length = density) .+ 1e-6
```

which will make $100\times100 = 10{,}000$ initial conditions. Plot the BoA as a heatmap just like they are presented in the lectures.

2.4 (5 points) So far your function `fg1` is finding groups that represent attractors. Extend it into a new function `fg2` in two ways, so that the new version also returns in addition

(a) the actual attractors in state space. To do this, you need to keep track of which initial conditions generated the features that became group "centers". For each initial condition that became a group center, store its trajectory in memory.

(b) the BoA fractions of each attractor (percentage of initial conditions that converged to each attractor).

Apply this extended function to the example system of the previous task. Plot the basins of attraction, and then on top of them scatter plot the found attractors. Assign a label to each of the plotted attractors, where the name of the label is their BoA fractions (rounded to 2 decimal digits) and display the labels as well.

Hint: make sure the colours of the attractors and the colours of their basins match, otherwise you will lose points! You probably want to add `strokewidth = 0.5, strokecolor = :black` *to the* `scatter` *function.*

2.5 (3 points) Consider the Duffing oscillator, given by:

$$\begin{aligned} \dot{x} &= v \\ \dot{v} &= -x - x^3 - \delta v + f\cos(\omega t). \end{aligned} \tag{3}$$

This non-autonomous dynamical system is a paradigmatic model for nonlinear forced oscillators. The term $f\cos(\omega t)$ represents a time-dependent driving force acting on the oscillator. The driving period is $\tau = 2\pi/\omega$ and the starting parameters of the model are $\delta = 0.1, f = 0.45, \omega = 2.15$. While formally the system is three-dimensional (because of the time forcing), you will visualise everything on the two-dimensional state space of $x - v$.

This dynamical system has three coexisting attractors (for the starting parameters). Similarly with Task 2.1, show a plot of at least three trajectories, each going to a different attractor. Ensure that every time you integrate a new trajectory for this system, you start the integration at time $t = 0$ (this is done automatically by the `trajectory` function). *Hint: always use sampling time of* $\Delta t = \tau/20$ *when evolving trajectories for this model.*

2.6 (3 points) Given the previous task, decide what `featurizer, r` will be suitable for this system. Then, use your `fg2` function to calculate and plot the basins of attraction (BoA) of the Duffing oscillator with the parameters of Task 2.5. To do so, initialize a grid of initial conditions $x \in (-2.5, 2.5)$ and $v \in (-2.5, 2.5)$. Use a total of 100 values in both ranges, which gives you $100\times100 = 10{,}000$ initial conditions. This time, do not plot the attractors on top of the BoA plot. Instead, making a second panel to the right of the BoA panel and visualize the attractors there on the $x - v$ plane.

Hint: for the Duffing model, always use 100τ *for* `T`, 200τ for `Ttr` and $\tau/20$ for `Dt`.

2.7 (2 points; hard) Now its time to put pressure on the `featurizer` you have made! Update the system parameters as follows: $\delta \to 0.2, f \to 3.4, \omega \to 1$ (don't forget to update `T, Ttr, Dt` according to the new period). In this scenario the system again has three attractors. Unfortunately, all attractors "intersect" with each other on the $x-v$ plane that you use for visualisations (remember, this is possible, because the system is non-autonomous). It is likely that the featurizer you have created so far won't correctly separate all three attractors (and if it does, then you've solved this Task for free!). Repeat the process of Task 2.5, look at the resulting trajectories, and identify the three unique attractors of the system. Comment on their nature: are they fixed points, periodic orbits, chaotic attractors, and how many of each?

Hint: for this task integrating trajectories with a transient time of 10 periods is enough to converge to the respective attractors.

2.8 (2 points; hard) Consider now the following function, which is applied to a trajectory `X`:

```
h(X) = length(unique([round.(p; digits = 1) for p in X])) / length(X)
```

This function is very useful in distinguishing the attractors of the system. Rationalise why in your coursework solutions. Then, use `h` in an updated featurizer, change the limits of your grid from (-2.5, 2.5) to (-5, 5), and produce a BoA plot, with the attractors in the right panel, as in Task 2.6. You should find that one attractor has basin fractions of ≈ 0.5 and the other two have ≈ 0.25 each.

3 Global continuation and bifurcations [25 points]

Most tasks of sections 3 and 4 require you to have solved correctly Task 2.4 for the predator prey model of Eq. (2). Compare your output with the result of using DynamicalSystems.jl directly to ensure it is correct. If you believe you have not succeeded in task, you can instead use the function `pp_bmap` provided at the end of this script to map initial conditions of the predator prey model into their basins/attractors labels.

You MUST disclose using this function after the header of section 3, and you MAY NOT use it in section 2!!!

3.1 (8 points) Continue from Task 2.4 with the predator prey system of Eq. (2). Perform a global continuation for this system while varying r from 0.2 to 3.0 using 50 values this range. Use the ASCM global continuation algorithm described in the lectures and ignore the "seeding" component. For all parameter values randomly sample 1000 initial conditions in the ranges of Task 2.4. To create the global continuation code you need to re-use the function
`fg2(ds, featurizer, ics; kw...)` with the same featurizer as in Task 2.4. For the matching step of the algorithm, use the centroid distances as discussed in the lectures. *The centroid of a state space set* `A` *is just* `mean(A)`.

Visualize your continuation by plotting four quantities versus the r parameter: 1) the BoA fractions, 2) the maximum of the N coordinate, 3) the maximum of the P coordinate, 4) the standard deviation of the N coordinate of the attractors (use a different Axis for each quantity). Ensure that properties that correspond to the same attractor are plotted with the same colours as in the lecture slides.

3.2 (4 points) Looking at the continuation plots, infer the bifurcation type that occurs at at $r \approx 1.5$ from the options we discussed in the lectures. Then, without doing the actual calculation, discuss what analytic (pen-and-paper) calculations you would definitively demonstrate that your answer is correct.

3.3 (2 points) Consider the parameter curve given by: $r(\theta) = 1.2 - 0.5\cos(\theta), \delta(\theta) = 1.5 - 0.5\sin(\theta), \theta \in [0, 2\pi]$. This defines an ellipsis in the r-δ parameter plane. Perform a global continuation, by re-using the code of Task 3.1, but now versus this parameter curve as θ varies from $0 \to 2\pi$ (use 100 discrete values in this interval). Visualize the results like in the previous task versus θ as the x-axis.

3.4 (4 points) Since this parameter curve is closed (its end is also its beginning), it is guaranteed that the attractors you have found at $\theta = 2\pi$ are exactly the same as when you started the continuation at $\theta = 0$. However, with the simple matching scheme you are re-using from Task 3.1, the attractors will not be matched correctly because some attractors that existed when $\theta = 0$ have disappeared as $\theta > 0$, and then reappeared when $\theta \to 2\pi$. Enhance the matching part of your implementation so that it also considers vanished attractors when matching, as is discussed in the lectures.

3.5 (2 points) How many bifurcations occur during this continuation? Which ones are local and which are global?

3.6 (3 points) Apply your global continuation code of task 3.1 to Duffing oscillator of Task 2.5. You are provided a featurizer function at the end of this task. Set the Duffing parameters to $\delta \to 0.15, \omega \to 1$, and vary f along the range: `frange = 0.22:0.005:0.32`. Important: for matching, don't use the centroid distance described in the lecture slides, as during the continuation multiple attractors have very similar

centroids. Instead, use as matching distance the Euclidean distance of the vectors composed of [x-mean(A), x-span(A)] for a trajectory A, with x-span being the maximum minus the minimum of the variable x and x-mean the mean of the variable x.

Visualize the result of the global continuation by plotting versus the f parameter: 1) the basins fractions, 2) the mean of the x coordinate of the attractors, 3) the span of the x coordinate of the attractors, 4) the value of `h` of the attractors.

Hint: you can use the following function as your featurizer:

```
function provided_featurizer(X)
    x = X[:, 1]
    xmean = mean(x)
    xspan = maximum(x) - minimum(x)
    h = length(unique([round.(p; digits = 1) for p in X])) / length(X)
    return SVector(xmean, xspan, 2h)
end
```

and a threshold value of 0.5.

3.7 (2 points; hard) At which f value during the continuation a chaotic attractor emerges and remains stable for a wide range of f? Justify your answer.

4 Critical transitions and nonlocal stability [25 points]

In this section we perform analysis of critical transitions that can occur to the predator prey model of Eq. (2).

4.1 (1 points) Focus at the continuation plot from Task 3.1 where $r \in [0.2, 3.0]$ is varied. Identify how many bifurcation tipping events can occur for this predator prey model and list the approximate values of r they occur at. Then, comment on their irreversibility: how much do you have to vary r to "recover" from this b-tipping (within the allowed range)?

4.2 (4 points) Set the system parameters $r \to 1.25, \delta \to 2.2$. Using the same code as in Task 2.4, calculate and plot the BoA and the attractors. Use the BoA you have estimated to calculate the minimal critical shock (MCS) for the coexistence ($N > 0, P > 0$) fixed point attractor of the system using the technique discussed in the lectures that utilizes an existing BoA. To achieve this, you need a function that calculates distances between two points on the state space of the system (also known as a "metric"). Use the following:

```
function generate_metric(Nweight, Pweight)
    function metric(u, v)
        sqrt( ((u[1] - v[1])/Nweight)^2 + ((u[2] - v[2])/Pweight)^2 )
    end
    return metric
end
mymetric = generate_metric(15, 0.03)
```

Calculate and plot the MCS on top of the existing basins plot.

4.3 (1 points) The metric you are creating essentially attributes a "cost" to changing the prey or predator populations via two weights. The smaller the weight the more costly it is to change the corresponding species. At the moment these costs are just the state space extent that we are exploring (which roughly correspond to maximum environmental capacity of each species). Under fixed `Nweight = 15`, find out the range of values of `Pweight` where the MCS switches from being N-dominated to being P-dominated. (such analyses can be used to inform interventions in conservation efforts)

4.4 (3 points) Set now $r \to 2.0$ and make a new plot of basins of attractions and attractors. The coexistence state is now a limit cycle. Focus on this attractor. Sample equi-temporal 50 points along the limit cycle by evolving a point on the cycle for one period with sampling time $\delta t = \Pi/50$ where Π is the period of the limit cycle, estimated approximately from a timeseries plot of any of the variables on this attractor.

Set `Pweight = 0.05`. Calculate the MCS for the 50 points along the limit cycle. Plot all 50 MCSs as arrows on top of the existing plot you have with the basins and the attractors.

4.5 (1 points) Report the magnitude and direction of the average minimal critical shock. Compare this magnitude with the average of the magnitudes of the individual minimal critical shocks for each point on the limit cycle. Explain the difference between the two. *Hint: to correctly calculate magnitudes you must utilize the custom metric you defined to calculate the MCS!*

4.6 (2 points) The inverse of the MCS magnitude is a measure of susceptibility to tipping. Calculate this for the 50 sampled points and normalize the susceptibility by the maximum susceptibility over these 50 points. Plot this susceptibility versus the *phase* of each point.

To calculate the phase you can use the following naive approach

```
function naive_phases(X::StateSpaceSet, scaling)
    center = mean(X)
    xs = X[:, 1] .- center[1]
    ys = X[:, 2] .- center[2]
    return atan.(scaling .* ys, xs)
end
```

with `scaling = Nweight/Pweight`.

Hint: you will re-use these phases and susceptibility a later Task; give them characteristic names!

4.7 (2 points) Consider the following two parameters: $r_1 = 1.6, r_2 = 2.55$. For each, calculate the BoAs. Plot the two BoAs (and their attractors) in two panels side by side. Assume that the system starts at $r = r_1$ and in the limit cycle state. If we are switching r between r_1, r_2, what tipping events can this system undergo?

4.8 (2 points) Identify which points of the limit cycle at $r = r_2$ are basin transferable. Their portion is a quantity ζ. Report the value of ζ.

4.9 (6 points) Perform now some randomized simulations (often also called Monte Carlo simulations). Define a function `random_r_sim(ds, m, n, metric)` with `ds` a dynamical system instance (which is the predator prey model you've been analysing this whole time), and `m, n` are integers, and `metric` the metric you have been using since Task 4.4. The function `random_r_sim` performs `m` repetitions of the following randomized experiment. In the experiment, the dynamical system is reset to the initial condition $[5.0, 0.01]$ and the parameter is set to $r = 1.6$. A random time is sampled from the distribution `randomtime() = 13*(2 + rand())` and the system is evolved for that time. Then, the following process repeats for `n` times: [set $r \to 2.55$; sample random time; evolve the system for that; set $r \to 1.6$; sample random time; evolve the system for that]. At the end of the `n` repetitions, it is checked if `condition = metric([0, 0], u) < 1e-6`, where `u` is the current system state. After `m` experiments, the function returns the portion of experiments for which the checked condition was true.

Run this function several times and output its result 5 times always using `m = 5, n = 5`.

4.10 (3 points) Consider now that `m` $\to \infty$. Provide an analytical expression (mathematical notation, not code!) that expresses the result of `random_r_sim(ds, Inf, n)` as a function of ζ, n, and justify how you arrived at the expression (i.e., why these randomized simulations yield such outcome given this `condition` definition). Show the numeric output for `m = 1000` and `n = 1, 5, 10` and discuss whether it serves as numerical proof for your analytical expression.

Hint: the probability of having at least one success in an n-times-repeated Bernoulli trial with success probability p is $P = 1 - (1 - p)^n$.

Here is the function `pp_bmap` that you can use in sections 3 and 4.

```
function pp_bmap(ds::DynamicalSystem, ics)
    pow = 4
    grid_recur = (
        range(0, 15^(1/pow); length = 100).^pow,
        range(0, (30e-3)^(1/pow); length = 100).^pow,
    )
    bmap_recur = BasinMapRecurrences(ds, grid_recur;
        consecutive_recurrences = 4000, consecutive_attractor_steps = 10,
        Δt = 0.2, stop_at_Δt = true, store_once_per_cell = false,
    )
    labels = map(bmap_recur, ics)
end
```

(this function only works for the predator prey model of Eq. (2), not the duffing oscillator)